

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**  
 Department of Computer Science and Engineering  
**ASSESSMENT TOOL 3 – Mock Interview**

|                  |                                                                                                                                                                |               |                                    |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------------------------|
| Course Code:     | CSA06                                                                                                                                                          | Course Title: | Design and Analysis of Algorithms  |
| CO Assessed:     | CO5 – Naturalization (BL5, BL6)                                                                                                                                | Assessment:   | Assessment Tool 3 – Mock Interview |
| Weightage:       | 30% of CIA                                                                                                                                                     | Total Marks:  | 100                                |
| CO5 Statement:   | Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving. |               |                                    |
| Student Name:    | Sasikumar Megha                                                                                                                                                | Reg. No.:     | 192471088                          |
| Section / Batch: | 3 <sup>rd</sup> CS & BS                                                                                                                                        | Date:         |                                    |

**Instructions to Students**

- Attend the mock interview individually and answer all technical questions clearly.
- Explain the logic, algorithm design steps, and complexity analysis wherever required.
- Demonstrate understanding of backtracking, recursion, and combinatorial problem-solving approaches.
- Differentiate clearly between P, NP, NP-Hard, and NP-Complete problems with suitable examples.
- Maintain clear communication, confidence, and logical presentation throughout the interview.

| Question Type  | Focus Area                                                                                                                                  | Questions |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Mock Interview | Demonstrate problem-solving skills in backtracking techniques and complexity classes through technical discussion and algorithmic analysis. | Q1- Q10   |

**SECTION A – Mock Interview**

(95) ✓

**Code of Conduct**

I, Sasikumar Megha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Megha

# RUBRICS FOR CALCULATION

| Criteria              | Wt. | Level 4 – Exemplary                                                               | Level 3 – Proficient                             | Level 2 – Developing                               | Level 1 – Beginning                   |
|-----------------------|-----|-----------------------------------------------------------------------------------|--------------------------------------------------|----------------------------------------------------|---------------------------------------|
| Technical Knowledge   | 25% | Demonstrates strong and accurate subject knowledge with in-depth understanding    | Shows good knowledge with minor gaps             | Basic knowledge with noticeable errors             | Lacks fundamental technical knowledge |
| Problem Solving       | 25% | Solves problems logically and applies concepts effectively in real scenarios      | Applies concepts with moderate accuracy          | Limited problem-solving ability; requires guidance | Unable to solve or apply concepts     |
| Analytical Thinking   | 15% | Analyzes questions critically and provides well structured, logical answers       | Provides reasonable analysis with some structure | Superficial or partially structured responses      | No analytical thinking demonstrated   |
| Communication         | 15% | Communicates clearly, confidently, and professionally with proper terminology     | Communicates well with minor hesitation          | Communication lacks clarity or confidence          | Unable to communicate effectively     |
| Confidence            | 10% | Displays high confidence, positive attitude, and professional behavior throughout | Shows good confidence with minor issues          | Low confidence; inconsistent behavior              | Lacks confidence and professionalism  |
| Response to Questions | 10% | Responds accurately, promptly, and elaborates with relevant examples              | Responds correctly but with limited depth        | Partial responses; needs prompting                 | Unable to respond appropriately       |

## Marks Evaluation:

| Criteria              | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|-----------------------|-----|---------------------|----------------------|----------------------|---------------------|
| Technical Knowledge   | 25% | ✓                   |                      |                      |                     |
| Problem Solving       | 25% | ✓                   |                      |                      |                     |
| Analytical Thinking   | 15% | ✓                   |                      |                      |                     |
| Communication         | 15% | ✓                   |                      |                      |                     |
| Confidence            | 10% | ✓                   |                      |                      |                     |
| Response to Questions | 10% | ✓                   |                      |                      |                     |
| Total Marks (100):    |     | 95                  |                      |                      |                     |

| Faculty In-charge             | Course Coordinator | Head of Department |
|-------------------------------|--------------------|--------------------|
| Name: <u>P. Srinivasan</u>    | Name: _____        | Name: _____        |
| Signature: <u>[Signature]</u> | Signature: _____   | Signature: _____   |
| Date: <u>31/8/24</u>          | Date: _____        | Date: _____        |

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Design and Analysis of Algorithms COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO5: Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.

(BL5,BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks: 25

Annexure A

Mock interview

Code of Conduct:

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

**Annexure A**

**Mock interview**

**Q90. Analyze when Backtracking is preferable over Dynamic Programming and when it is not.**

**Backtracking** is preferable when:

- The problem involves **finding all possible solutions**.
- Solutions must satisfy several **constraints**.
- Invalid solutions can be rejected early using **pruning**.
- Example: **N-Queens, Graph Coloring, Sudoku**.

**Dynamic Programming (DP)** is preferable when:

- The problem has **overlapping subproblems**.
- It has **optimal substructure**.
- We need an **optimal solution efficiently**.
- Example: **Knapsack, LCS, Matrix Chain Multiplication**.

**Comparison:**

| <b>Backtracking</b>          | <b>Dynamic Programming</b>           |
|------------------------------|--------------------------------------|
| Explores solution tree       | Stores previously solved subproblems |
| Uses pruning                 | Uses memoization/tabulation          |
| Often exponential            | Usually polynomial                   |
| Good for constraint problems | Good for optimization problems       |

**Conclusion:** Use backtracking for **constraint satisfaction and solution enumeration**; use DP when repeated subproblems allow computation to be reused.

**Q91. Explain why Graph Coloring is considered NP-Complete for  $k \geq 3$ .**

The **k-Coloring decision problem** asks whether the vertices of a graph can be colored using at most **k colors** such that adjacent vertices have different colors.

For  $k \geq 3$ , it is NP-Complete because:

1. **It belongs to NP:**  
Given a coloring, we can check every edge and verify that adjacent vertices have different colors in polynomial time:  $O(V + E)$ .
2. **It is NP-Hard:**  
The **3-Coloring problem** is a known NP-Complete problem. For any fixed  $k \geq 3$ , 3-Coloring can be transformed into k-Coloring.

Therefore:

**Graph k-Coloring = NP + NP-Hard = NP-Complete for  $k \geq 3$ .**

**Note:** 2-Coloring is different; it can be solved efficiently using **BFS/DFS** in polynomial time.

**Q92. Compare the complexity of Graph Coloring and Hamiltonian Cycle problems.**

| Problem           | Worst-Case Time                                        | Complexity Class           |
|-------------------|--------------------------------------------------------|----------------------------|
| Graph Coloring    | Exponential, approximately $O(k^V)$ using backtracking | NP-Complete ( $k \geq 3$ ) |
| Hamiltonian Cycle | Exponential, approximately $O(V!)$ using brute force   | NP-Complete                |

**Graph Coloring:**

For each vertex, up to  $k$  colors may be tried. Therefore, the search space can be:

$$k^V$$

**Hamiltonian Cycle:**

We may need to examine permutations of vertices:

$$(V - 1)!$$

**Conclusion:** Both are **NP-Complete** and can require exponential/factorial search in straightforward algorithms. Hamiltonian Cycle generally has a larger brute-force search space because of its factorial nature.

**Q93. Analyze the time complexity of generating all permutations of a set of size  $n$ .**

For  $n$  distinct elements, the number of permutations is:

$$n!$$

If generating each permutation takes  $O(n)$  time, then the total time is:

$$O(n \times n!)$$

For example:

- 3 elements  $\rightarrow 3! = 6$
- 4 elements  $\rightarrow 4! = 24$
- 5 elements  $\rightarrow 5! = 120$

A backtracking algorithm explores all permutations, so the **time complexity is  $O(n \cdot n!)$**  when the cost of constructing/outputting each permutation is included.

**Space complexity:**

Typically  $O(n)$  auxiliary recursion space, excluding the storage required for all output permutations.

```
genui{"learning_viz":{"type_id":"PERMUTATION_FORMULA","initial_values":{"n":5,"r":3}}}
```

**Q94. Explain why permutation problems inherently have factorial time complexity.**

Suppose there are  $n$  elements.

- First position  $\rightarrow n$  choices
- Second position  $\rightarrow n - 1$  choices
- Third position  $\rightarrow n - 2$  choices
- ...
- Last position  $\rightarrow 1$  choice

Therefore, the total number of arrangements is:

$$n(n - 1)(n - 2) \cdots 2(1) = n!$$

So any algorithm that **generates every permutation** must process at least  $n!$  different arrangements.

Hence, permutation generation inherently requires **factorial time**:

$$\boxed{\Omega(n!)}$$

If each permutation requires  $O(n)$  time to output, the total becomes:

$$\boxed{O(n \cdot n!)}$$

**Key point:** The factorial complexity comes from the **number of possible permutations**, not simply from the algorithm used.

#### Q95. Derive the worst-case complexity of subset generation using Backtracking.

For a set containing  $n$  elements, every element has **two choices**:

1. Include the element.
2. Exclude the element.

Therefore, the number of subsets is:

$$2^n$$

The backtracking recursion forms a binary decision tree:

- Level 0  $\rightarrow 1$  choice
- Level 1  $\rightarrow 2$  choices
- Level 2  $\rightarrow 4$  choices
- ...
- Level  $n \rightarrow 2^n$  choices

Thus, the number of generated subsets is:

$$\boxed{2^n}$$

If each subset requires  **$O(n)$**  time to construct/output, the total time is:

$$O(n2^n)$$

**Space complexity:** The recursion depth is n, so auxiliary space is:

$$O(n)$$